

LSHFed

Empirical Analysis of Locality-Sensitive Hashing Gradient Mapping in Robust Federated Learning

This report investigates the geometric properties and operational trade-offs of Locality-Sensitive Hashing Gradient Mapping (LSHGM) within the LSHFed framework (Cheng et al., AAAI-26). By systematically sweeping the random hyperplane projection count r in $[1, 256]$ across stochastic model gradients, controlled synthetic vectors, and dynamic federated trajectories on CIFAR-10, we evaluate the relationship between Euclidean and LSH Hamming distances. The findings reveal that LSHGM operates as a purely angular distance metric, completely invariant to gradient magnitude, resulting in weak global correlation with Euclidean distance ($\rho = -0.1345$ on real FL trajectories). However, we demonstrate that weak global correlation does not preclude high agreement in aggregator-selection decisions. Finally, we show that increasing projection counts beyond $r = 32$ introduces significant computational and memory overhead without yielding monotonic improvements in verification decisions.

Background

Federated Learning (FL) enables distributed clients to collaboratively train machine learning models while keeping raw data localized. To mitigate Byzantine attacks and malicious model poisoning, robust FL protocols rely on verification mechanisms to evaluate local updates against reference benchmarks.

LSHFed introduces Locality-Sensitive Hashing Gradient Mapping (LSHGM) to accelerate this verification phase. By applying r random hyperplane projections to high-dimensional gradient tensors, LSHGM converts parameters into compact binary hashes, replacing Euclidean distance calculations with bitwise Hamming distance operations.

While the original LSHFed formulation adopts a fixed projection hyperparameter (r), the underlying assumption that Hamming distance serves as an accurate surrogate for Euclidean distance across varying projection scales has not been thoroughly benchmarked. Selecting r represents a fundamental trade-off: higher projection counts theoretically improve distance estimation granularity, yet linearly scale matrix-projection runtime and hash storage overhead.

Experimental Sequence

This study systematically evaluates LSHGM across four sequential experimental phases:

- **Experiment 01 (Within-Model Correlation):** Baseline distance correlation on stochastic gradients extracted from a fixed ResNet-9 model.
- **Experiment 02 (Synthetic Gradient Correlation):** Controlled geometric isolation of vector magnitude versus directional perturbations.

- **Experiment 03 (Verifier Update-Distance Correlation):** Evaluation of LSHGM verification decisions across a live CIFAR-10 federated training trajectory.
- **Experiment 04 (Projection Count Tradeoff):** Resource-accuracy optimization sweeping r across frozen candidate-benchmark update pairs.

Experiment 01: Within-Model Gradient Correlation

1. Motivation / Objective

LSHFed uses random-hyperplane locality-sensitive hashing (LSH) to represent model gradients as binary hashes. The distance between two gradients can then be estimated using the Hamming distance between their hashes instead of directly comparing the full gradient tensors.

This experiment evaluates how accurately Hamming distance tracks the Euclidean baseline across varying projection counts of r ?

Prior to evaluating a full federated trajectory, this relationship is benchmarked in a static model setting to isolate gradient dynamics. A single model is kept fixed while multiple stochastic gradients are generated, eliminating variance from evolving parameters.

We systematically sweep the projection hyperparameter r . If metric divergence is driven primarily by insufficient projection density, scaling r should systematically increase correlation.

The experiment therefore asks:

- To what extent are Euclidean and LSH Hamming distances correlated for stochastic gradients derived from a static model state?
- Does increasing r systematically improve metric correlation?

2. Methodology

Experimental setup

The experiment uses the CIFAR-10 dataset and a ResNet9-style model. The model is kept at one fixed state while stochastic gradients are collected.

For each of 100 gradient samples, the same model parameters are used, while the training data and stochastic behavior provide the variation between gradients.

The gradient used for the experiment is the model parameter tensor with shape $512 \times 512 \times 3 \times 3$.

This gives 100 gradients and therefore $(100 \text{ choose } 2) = 4,950$ unique gradient pairs.

Distance comparison

For two gradients g_i and g_j , the reference distance is their Euclidean distance:

$$d_E(g_i, g_j) = \|g_i - g_j\|_2$$

The LSHFed hashing procedure first flattens the gradient using the same 2D representation used by the authors' implementation. Random projection vectors are then applied and the sign of each projection produces a binary hash.

The Hamming distance between two hashes is used as the LSH-based distance:

$$d_H(h_i, h_j) = \sum_k \mathbb{I}[h_{i,k} \neq h_{j,k}]$$

The experiment compares these two distances over all 4,950 gradient pairs.

Projection sweep

The tested projection counts are r in $\{1, 2, 4, 5, 8, 16, 32, 64, 128, 256\}$.

A fixed bank of 256 random projection vectors is generated, and smaller values of r use prefixes of the same bank. This keeps the projection randomness fixed while r changes.

For every r , Pearson and Spearman correlations are calculated between the Euclidean and Hamming distances.

3. Results and Interpretation

The Euclidean–Hamming relationship is weak across the entire sweep.

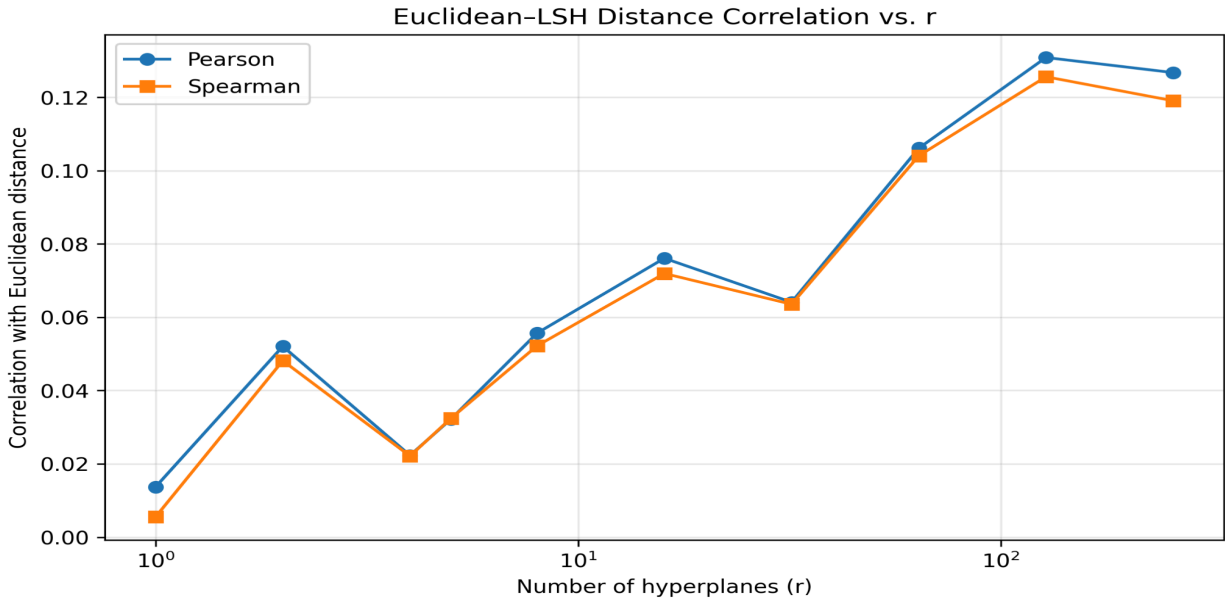


Figure: Correlation of Euclidean and LSH-Hamming distance recorded from within-model gradients.

At $r = 1$, the Pearson correlation is only about 0.014. Even at $r = 256$, it reaches only about 0.131. Spearman correlation follows the same pattern, reaching only about 0.126 at $r = 256$.

Even though the correlation does increase from 0.014 to 0.131 due to increasing r , the highest correlation reached at $r = 256$ is still very low.

These results motivate Experiment 02, which utilizes synthetic gradient vectors to isolate the independent geometric impacts of magnitude scaling versus directional variance.

Experiment 02: Synthetic Gradient Correlation

1. Motivation / Objective

Experiment 01 found that Euclidean distance and LSH Hamming distance have very low correlation for stochastic gradients generated from the same model, and that increasing r results in a meager improvement of the correlation.

The two distance measures do not respond to gradient differences in exactly the same way. Euclidean distance changes when either the magnitude or direction of a gradient changes. Random-hyperplane hashing, on the other hand, records the signs of projections and is therefore only sensitive to directional changes.

Experiment 02 uses synthetic gradients to isolate these effects.

It asks:

- What happens when two gradients differ only in magnitude?
- What happens when their direction is changed?
- When gradient differences are generated through controlled directional perturbations, does increasing r improve Hamming–Euclidean correlation?

The purpose is not to reproduce the full LSHFed training process. It is to understand the geometry behind the result from Experiment 01.

2. Methodology

Synthetic gradient representation

A base gradient B with shape 64×128 is generated. A fixed bank of 256 random projection vectors is used for the LSH calculations.

The hashing implementation follows the relevant LSHFed conventions for flattening, projection, binary hashing, and Hamming-distance calculation.

Case 1: Magnitude-only changes

The first test creates gradients by scaling the same base vector:

$$g_s = s \cdot B$$

The direction of g_s is unchanged as s varies.

Euclidean distance changes because the vector magnitude changes. However, for positive s , every projection keeps the same sign. The resulting hashes are therefore identical and their Hamming distance remains zero.

This isolates the part of Euclidean distance that LSH hashing does *not* capture.

Case 2: Directional changes

The second test adds a random perturbation:

$$g_{\alpha} = B + \alpha \cdot N$$

where N is a random perturbation and α controls its strength.

As α increases, the gradient changes direction. This can cause individual random projections to cross zero, producing different hash bits.

The Euclidean and Hamming distances are recorded as the perturbation strength increases.

Case 3: Synthetic gradient population

The final test generates 200 gradients of the form

$$g_i = B + \alpha_i \cdot N_i$$

using independently generated perturbations and perturbation strengths.

All $(200 \text{ choose } 2) = 19,900$ gradient pairs are compared.

The experiment first evaluates the complete 256-projection hashes and then repeats the correlation calculation for r in $\{1, 2, 4, 5, 8, 16, 32, 64, 128, 256\}$.

The same projection bank is used throughout the sweep so that changing r does not introduce a new source of randomness.

3. Results and Interpretation

Magnitude changes are not reflected in Hamming distance.

Scale	Euclidean Distance	Hamming Distance
1.00	0.00	0
1.01	0.90	0
1.05	4.51	0

1.10	9.03	0
1.50	45.14	0
2.00	90.28	0

Euclidean distance increases as the scale changes, while Hamming distance remains zero.

This shows that two gradients can be increasingly far apart in Euclidean distance while having exactly the same random-hyperplane hash, as long as their direction does not change.

Directional changes are reflected in Hamming distance

When directional perturbations are introduced, Hamming distance starts changing along with Euclidean distance.

Noise Level (α)	Euclidean Distance (dE)	Hamming Distance (dH)	Normalized Hamming
0.00	0.00	0	0.0000
0.01	0.90	93	0.0028
0.05	4.52	487	0.0149
0.10	9.05	987	0.0301
0.25	22.62	2,508	0.0765
0.50	45.24	4,783	0.1460
1.00	90.48	8,113	0.2476

This shows that the LSH representation is responding to directional differences that cause gradients to cross projection hyperplanes.

Together, the first two tests explain an important part of the mismatch seen in Experiment 01: Euclidean distance includes magnitude differences that the sign-based hash does not directly represent.

Synthetic population

The relationship is very different when the synthetic gradient population is generated through controlled directional perturbations.

At $r = 256$:

- Pearson correlation = 0.9944
- Spearman correlation = 0.9964

Correlation also improves steadily as r increases:

r	Pearson	Spearman
1	0.8830	0.8659
2	0.9295	0.9171
4	0.9660	0.9583
5	0.9722	0.9670
8	0.9818	0.9786
16	0.9872	0.9859
32	0.9910	0.9913
64	0.9927	0.9940
128	0.9938	0.9957
256	0.9944	0.9964

So, in this controlled synthetic setting, increasing r gives a progressively better approximation of the Euclidean ordering.

This indicates that the Hamming–Euclidean correlation is only high when the gradients are dominated by directional perturbation. For gradients that are dominated by differences in magnitude only, the correlation and subsequently the malicious client filtering decisions made by Euclidean and Hamming can differ.

The next step is therefore to move from synthetic gradients to actual model updates produced by federated training. Experiment 03 tests the distance relationship in an LSHFed-style federated trajectory and, more importantly, checks whether the two distances lead to the same aggregator-selection decision.

Experiment 03: Verifier Update-Distance Correlation

1. Motivation / Objective

The first two experiments looked at the relationship between Euclidean and LSH Hamming distance in controlled settings.

Experiment 01 found low correlation for stochastic gradients from a fixed model. Experiment 02 showed that the relationship can be much stronger when gradient differences are generated through controlled directional perturbations.

The remaining question is what happens on actual updates produced by federated training.

This experiment therefore implements an LSHFed-style federated-learning trajectory with multiple local trainers and aggregators. Instead of only asking whether the two distances are correlated, it also asks the more practical question:

When Hamming distance is used to choose the next aggregator, does Euclidean distance choose the same aggregator?

This distinction matters because LSHFed ultimately uses the distance comparison to make a selection. A weak global correlation does not necessarily mean that the selected aggregator will be different.

2. Methodology

Federated setup

The experiment uses a 10,000-sample subset of CIFAR-10 with:

- 5 Local Trainers
- 2 Aggregators
- 2,000 samples per Local Trainer
- 1 local training epoch
- 8 federated rounds
- $r = 5$ for the LSH distance calculation

The aggregators are formed as:

$AG_0 = \{LT_0, LT_2, LT_4\}$

and

$AG_1 = \{LT_1, LT_3\}.$

Each round performs local training and aggregation, then compares the candidate aggregator updates with the current benchmark.

Round 0 has no previous benchmark, so AG0 is used to initialize it. Rounds 1 through 7 then provide two candidate-vs-benchmark comparisons per round, giving 14 distance observations in total.

Federated trajectory

The pipeline is:

1. Local data
2. Local training
3. Local Trainer updates
4. Aggregator updates
5. Compare candidate aggregators with benchmark
6. Select the Hamming-distance winner
7. Update the global model
8. Next round

The Hamming-based selection is the one used to continue the actual trajectory.

Model state and distance calculation

The complete model state is passed between rounds so that the federated training process continues from the selected global model state.

The distance comparison itself is performed on the trainable parameter updates.

For two candidate updates U and V , the Euclidean reference distance is

$$d_E(U, V) = \sqrt{\sum_k \|U_k - V_k\|_2^2}$$

where the sum is over the trainable parameter tensors.

The same updates are flattened and passed through the LSHFed-style random-hyperplane hashing procedure. Their Hamming distance is then

$$d_H(H_U, H_V) = \sum_k \mathbf{1}[H_{U,k} \neq H_{V,k}]$$

For every round, both distances are recorded for each candidate aggregator.

Two types of comparison are then made:

1. Pearson and Spearman correlation over all observations.
2. Winner agreement, checking whether Hamming and Euclidean choose the same aggregator within each round.

3. Results and Interpretation

The overall distance correlation is weak:

- Pearson = -0.1345
- Spearman = -0.4769

So the actual federated updates do not show the strong positive Hamming–Euclidean relationship observed in the controlled synthetic experiment.

However, the selection behavior tells a different story. Hamming and Euclidean choose the same aggregator in 6 of the 7 rounds, giving: Winner Agreement = $6 / 7 = 85.7\%$

There is one clear disagreement. In round 4:

- Hamming distance selects AG1
- Euclidean distance selects AG0

One important observation is therefore that weak global distance correlation does not translate directly into frequent disagreement in the aggregator-selection decision.

Correlation measures how the two distances vary across all candidate/benchmark pairs. Winner agreement only asks which candidate is smaller within each round. Those are different properties.

This result changes the focus of the next experiment. Rather than looking only at correlation, Experiment 04 varies the number of projections r and measures winner agreement together with the computational and storage cost of the hash representation.

Experiment 04: Projection Count Tradeoff

1. Motivation / Objective

The previous experiment 03 moved from synthetic gradients to an actual federated-learning trajectory. It found weak overall Hamming–Euclidean correlation, but the two metrics still selected the same aggregator in 6 of 7 rounds.

This raises a practical question: how does the choice of r affect the actual aggregator-selection decision?

A larger number of random hyperplanes gives a longer hash, which can provide a better representational capacity and a more granular resolution. This may change how closely the Hamming distance follows Euclidean distance. However, it requires more projection operations. Therefore, more projections have both a potential benefit and a cost.

Experiment 04 studies this tradeoff by sweeping

r in $\{1, 2, 4, 5, 8, 16, 32, 64, 128, 256\}$.

For each value of r , we measure:

- correlation (Pearson and Spearman),
- winner agreement with Euclidean distance,
- hash-generation time,
- Hamming-comparison time,
- hash representation size.

The main goal is not to assume that a larger r is better, but to see what actually happens on the federated trajectory.

2. Methodology

Fixed federated trajectory

Running a new federated trajectory for every r would change more than the projection count. A different r could lead to a different aggregator choice, which would then produce different model updates in later rounds.

To isolate the effect of r , this experiment first generates one fixed federated trajectory using the baseline $r = 5$.

The candidate/benchmark update pairs from all 8 rounds are then saved and frozen. Every value of r is evaluated on those same pairs.

This gives the following pipeline:

1. Generate one LSHFed-style trajectory with $r = 5$
2. Record candidate/benchmark updates
3. Freeze pairs
4. Sweep $r = 1, 2, 4, \dots, 256$ on the same frozen updates
5. LSH hashing + Hamming distance
6. Winner agreement + correlation
7. Runtime + hash size

The federated setup follows Experiment 03:

- 10,000 CIFAR-10 samples
- 5 Local Trainers
- 2 Aggregators
- 2,000 samples per Local Trainer
- 1 local epoch
- 8 rounds
- 14 candidate/benchmark comparisons

The complete model state is still passed between rounds, while the distance calculations use the trainable parameter updates.

LSH projection banks

A master bank of 256 standard-normal projection vectors is generated for each required vector dimension. Each smaller r uses a prefix of the corresponding bank.

This keeps the projection vectors consistent across the sweep.

The hashing implementation retains the per-vector `torch.mv` loop used by the authors' implementation. This is intentional because runtime is one of the quantities being measured.

Hash size

Each additional projection adds another set of binary hash values. Therefore the representation size grows linearly with r .

For example:

- $r = 5$: 80,580 bits, about 10 KB

- $r = 256$: 4,125,696 bits, about 516 KB

3. Results and Interpretation

Winner agreement is not monotonic with r

The winner-agreement results are:

r	Winner Agreement
1	85.7%
2	85.7%
4	100.0%
5	100.0%
8	100.0%
16	100.0%
32	100.0%
64	85.7%
128	71.4%
256	71.4%

In this trajectory, $r = 4$ through $r = 32$ all produce 100% winner agreement with the Euclidean reference.

Increasing r beyond this range does not improve the decision result. In fact, agreement falls to 85.7% at $r = 64$ and 71.4% at $r = 128$ and $r = 256$.

So the experiment does not show a simple relationship where more projections always produce better aggregator decisions.

Correlation also changes non-monotonically

Pearson correlation varies substantially across the sweep:

r	Pearson	Spearman
1	0.0311	0.0725
2	0.2951	0.5604
4	0.4797	0.3890
5	0.6631	0.3978
8	0.5426	0.3319
16	0.3474	0.2308
32	0.5844	0.3934
64	0.6845	0.0681
128	0.7099	0.0769
256	0.6560	0.0945

The highest Pearson correlation occurs at $r = 128$, but that value does not give the highest winner agreement. This is another example of why distance correlation and aggregator-selection agreement should be treated as separate measurements.

Increasing r has a clear computational cost

Runtime increases strongly with the number of projections.

Average total time for hash generation plus Hamming comparison per pair is approximately:

r	Total Time (s)
1	0.0139
5	0.0527
32	0.3034
64	0.6939
128	1.3476

256	2.6662
-----	--------

The hash representation grows in the same direction, from roughly 10 KB at $r = 5$ to roughly 516 KB at $r = 256$.

Overall observation

For this controlled federated trajectory, the useful range is not simply "the largest possible r ." Moderate values from $r = 4$ to $r = 32$ achieve perfect winner agreement in this run, while larger values substantially increase computation and hash size without improving that decision metric.

The results should be viewed as a measurement of the tradeoff on this particular trajectory. There are only 14 frozen candidate/benchmark comparisons, so the experiment does not establish a universal best value of r .

What it does show is that increasing the projection count has a real cost, while the decision benefit is not guaranteed to increase with it.